COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Gate E: A Small 3D Engine

Integrated project specification and review

Acceptance matrix · Failure tests · Scoped measurements · Independent clean-build review

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Gate E: a small 3D engine

Original checkpoint requirement

After Volume 5. Build an engine demonstration containing a CPU reference renderer, one supported GPU backend where available, a fixed-step simulation, explicit entity/resource ownership, an asset cooker, an inspector and versioned save/load. A software-only execution path is acceptable while GPU access is unavailable, but untested GPU claims must remain excluded.

Deliver a fixed scene, source assets with provenance, build/cook commands, validation logs, headless tests, screenshot or image-difference evidence, frame-time data and a peak-memory report. The budget is chosen for the declared test machine; 60 fps is an example target, not a universal pass requirement. Test near-plane crossing, resize/minimize, stale handles, malformed assets and resource shutdown.

A second person must build from a clean checkout or extracted source tree. The learner must explain one CPU/GPU dependency and one quality/performance tradeoff. Using an existing engine for comparison is encouraged; using it in place of the central engine implementation changes the learning objective and needs a separate implementation project.

Deliver one coherent system

Use a small authored scene, not an ambitious game. A useful target is a floor, several distinct objects, a moving entity, two material settings and a visible resource/state inspector. Keep all geometry and source assets under version control or in the submitted source folder. Procedural assets are acceptable with provenance. Declare what your engine does and what it deliberately omits.

Reuse the five module projects as the vertical implementation path. Your CPU renderer is the reference; the GPU path implements equivalent scene/convention intent where available. Your fixed-step world produces renderable snapshots. Your cooker translates source assets into validated runtime data. Your inspector issues undoable commands against authoring state, and your save/load path restores a versioned scene without serializing raw addresses.

The supplied package includes smaller examples of several contracts, but **does not contain a completed Gate E engine**. Finishing, connecting and reviewing those systems is the learner's work. The included cooker creates a GEM1 mesh consumed by the CPU reference and a GET1 mip payload that is not yet sampled by that renderer. PBR, native GPU backend, live editor, async residency and engine-wide integration remain open implementation tasks.

Milestone sequence and exit evidence

Stage	Required deliverable	Exit evidence
E1: reference visibility	CPU image/geometry path	Byte tests, six-plane clipping, shared-edge/depth/UV proof
E2: chosen GPU route	Scene rendering with explicit ownership	Actual capability and validation log; readback; resize/stop cases

Stage	Required deliverable	Exit evidence
E3: image quality	Documented material and texture pipeline	Controlled reference/motion comparisons and scoped timing
E4: engine core	Headless world, identities, lifetimes and persistence	Fixed-step trace, stale-handle and graph tests, candidate-load rejection
E5: asset/editor integration	Cooker, residency policy, inspector, undo, save/load	Reproducible keys, memory accounting, reversible edit and load sequence
E6: reproducible review	Clean build/run and evidence report	Independent reviewer record, raw budgets, known limitations

A software-only E2 may be marked DEFERRED while GPU hardware is unavailable, with all GPU performance/correctness claims excluded. Do not quietly rewrite DEFERRED as PASS. The native API project remains valuable follow-on work when access is arranged.

Minimum repository contract

At the root, include README commands that work from a clean folder, a declared dependency/toolchain inventory, source and build scripts, source assets and provenance, a cooker command, a fixed scene/seed, automated tests and a run command. Keep generated outputs in a separate directory. Save source before collecting evidence so each measurement can be traced to a revision or archive digest.

The engine must run its world/rules tests without creating a window or GPU. Save files contain stable IDs, versioned fields and resource references rather than process addresses. Candidate parsing and validation happen before publishing the new world. Error paths produce understandable messages and nonzero command status where appropriate.

Functional acceptance matrix

Feature	Required proof	Not sufficient
CPU geometry	Visible near-plane crossing without divide artifacts; shared-edge ownership test	A screenshot of an unclipped triangle
GPU ownership	One producer/consumer dependency and a retirement timeline explained from actual run	CPU queue-submit duration called GPU time
Simulation	Same step input gives stated tick sequence; overload policy observable	A fixed delta constant with an unbounded catch-up loop
Entity identity	Stale handle rejected after destroy/ reuse	Pointer still happens to point into allocated memory

Feature	Required proof	Not sufficient
Asset cook	Same inputs/settings/dependencies/ tool version reproduce payload and key	Same filename or timestamp
Residency	Old plus new/staging/temporary allocations fit policy or trigger fallback	Compressed file size alone
Inspector/undo	A validated edit, undo, redo and branch-after-undo sequence	GUI widgets directly mutating simulation arrays
Persistence	Version migration and malformed-candidate rejection preserve current world	Dumping in-memory struct bytes
Clean execution	Another person builds, cooks and runs with only the README	Author's existing build folder

Failure-path test matrix

Perform and retain these cases. A test is more useful when it records the expected failure before execution.

1. Move a triangle through each clip plane; include near-plane crossing and a degenerate triangle. Confirm no nonfinite output or uncontrolled allocation.
2. Resize repeatedly, minimize or force zero drawable extent, restore and close during initialization/ rendering. Keep size-dependent resources and history coherent. For a software-only headless route, document which window tests are inapplicable, then test equivalent extent/initialization contracts without inventing GUI execution.
3. Delay completion or reduce available frame contexts. Attempt reuse before last use completes; the engine must wait, reject or defer safely.
4. Destroy and recreate an entity slot, then access the old handle. Reject it without corrupting the new entity. State the generation wraparound policy.
5. Supply malformed asset bytes, unsupported version, impossible count, nonfinite position, duplicate stable ID and truncated save data. Preserve the previous valid manifest/world.
6. Start loading an optional resource, cancel the request, and allow the old worker completion to arrive. Do not publish into a reused request identity or free a still-referenced allocation.
7. Set a budget lower than the requested old+new+staging peak. Record a controlled fallback, admission rejection or eviction plan rather than an unexplained memory spike.
8. Edit, undo, redo, then undo and create a new edit. Verify redo history invalidation and no history change after an invalid edit.
9. Interrupt initialization after each subsystem creation and stop during a load. Demonstrate cleanup of exactly the resources actually created.
10. Rebuild/cook twice from two fresh directories, change a dependency or setting and confirm targeted invalidation. Corrupt a candidate input and show that the last valid published manifest remains intact.

The provided unit tests cover selected pieces, not this entire integrated matrix. Keep an explicit test-to-requirement mapping so an untested box remains visible.

Measurement plan

Choose a declared machine, fixed build mode, fixed scene/seed and output resolution. Choose a frame target appropriate to that route and justify it. At 60 frames/s the nominal interval is 1000/60 = 16.667 ms, but 60 is an example, not a universal gate requirement. Do not combine simulation, submission, GPU and presentation measurements under a misleading single label.

Record warmup policy, sample count, distribution and quantile method. Retain raw per-frame values and report median, p95, p99 and maximum alongside average. A handful of timings does not establish a stable tail. Identify whether your clocks measure CPU wall time, GPU timestamps or end-to-end presentation. Do not subtract timestamps from unrelated clock domains without calibration.

For memory, distinguish logical data bytes, tracked allocation bytes, process resident set, GPU residency estimates and the measurement tool's observable scope. Report observed peak during asset replacement/loading as well as steady state. Include source/decode/staging/scratch/old+new residency and frame resources according to the architecture. Do not add overlapping measurements as if they were disjoint allocations.

The provided evidence/ timings are local single-run CPU diagnostics. They are not budgets for your future engine, a machine recommendation or commercial performance claims. Repeat your own measurements under controlled conditions.

Suggested review rubric

Category	Weight	Passing evidence
Correctness and failure containment	30	Reproducible boundary/failure tests; no unresolved critical corruption
Ownership and architecture	20	Explicit identities, consumer lifetimes, subsystem boundaries
Rendering and numerical reasoning	20	Conventions, visibility, material/filter assumptions, controlled comparison
Build/assets/persistence	15	Reproducible cook/build, stable schema, atomic candidate publication
Measurement and explanation	15	Scoped raw evidence, honest limitations, independent review

A suggested pass is at least 80/100 with every mandatory acceptance area demonstrated and no unresolved correctness-critical defect. A high score cannot waive malformed-load corruption, unsafe resource reuse or fabricated validation. Document the software-only exception explicitly rather than awarding GPU points for unexecuted work; the reviewer should adjust the route's agreed evidence before assessment.

Independent review and oral defense

Give another person the extracted source tree or a clean checkout, not your build directory. Ask them to follow README from a fresh output folder. Record OS/toolchain, all commands, what failed before assistance and any changes necessary. Complete templates/REVIEWER_SIGNOFF.md. The package authoring run is not this review.

During the demonstration, explain one CPU/GPU producer-consumer dependency or, on the declared software-only route, the pending GPU design and one actually tested CPU ownership dependency. Explain one quality/performance tradeoff with controlled evidence. Trace one invalid load from source bytes through validation to rejected publication. Demonstrate an edit/undo/save/load sequence and identify which state belongs to authoring versus simulation preview.

Conclude with a short postmortem: the hardest failed hypothesis, the test that exposed it, the measured limitation and the next change you would make. The next curriculum volume, Game Systems and Production, should build on a small reliable system rather than a large unreviewed prototype.